AI Agent 时代：从"能用"到"好用"，基础设施不可缺

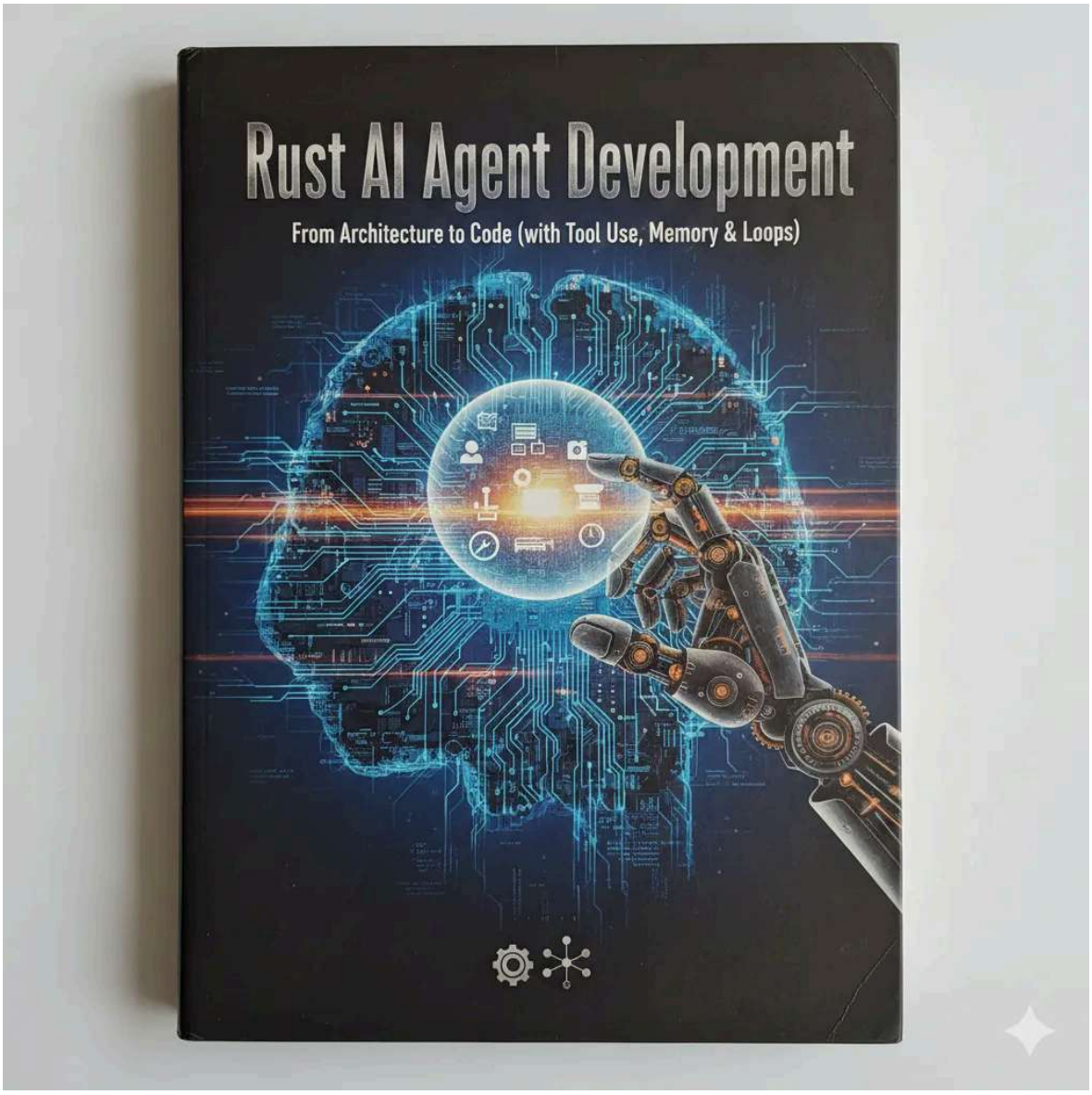
Rust语言中文社区 2026年3月7日 21:02 加拿大

以下文章来源于猿禹宙 ，作者猿禹宙

猿禹宙

一位热衷于 Web3 与 Rust 领域的数字码农，致力于开源社区的发展。 衷心希望这里的...

> *在大模型能力飞速迭代的今天，Agent 已成为 AI 应用的新范式。*
> *但 Agent 真正落地生产，需要的不只是模型，更需要一套扎实的基础设施。*



前言

当我们真正着手开发一个 Agent 应用时，会发现：

- **多模型适配**：OpenAI、Anthropic、Google 各有各的 API 风格
- **工具调用**：Function Calling 的实现细节繁琐且容易出错
- **上下文管理**：长对话的 token 消耗是个无底洞
- **成本追踪**：不同模型的定价策略各异，账单难以预测

这些问题让我们意识到：**Agent 时代，基础设施不可缺失。**

于是，我们开源了 **agent-io** —— 一个用 Rust 编写的 Agent SDK，希望为开发者提供一块坚实的基石。

为什么需要一个 Agent SDK?

痛点一：多模型适配的复杂性

每个 LLM 提供商都有自己独特的 API：

```
# OpenAI 风格
response = client.chat.completions.create(
    model="gpt-5.2",
    messages=[{"role": "user", "content": "..."}],
    tools=[...]
)

# Anthropic 风格
response = client.messages.create(
    model="claude-opus-4-6",
    messages=[{"role": "user", "content": "..."}],
    tools=[...]
)

# Google 风格
response = client.generate_content(
    contents=[{"role": "user", "parts": [...]}],
    tools=[...]
)
```

不同 API 的参数结构、响应格式、流式处理方式都不尽相同。如果业务需要切换或同时使用多个模型，适配工作量巨大。

痛点二：工具调用的繁琐细节

Tool Calling 听起来简单，实际实现却有很多坑：

- 如何定义工具的 JSON Schema？
- 如何处理工具调用的并发？
- 如何管理工具返回结果的生命周期？
- 如何实现工具之间的依赖注入？

痛点三：上下文爆炸

Agent 在执行复杂任务时会进行多轮迭代，每轮都会积累消息。如果不加控制，token 消耗会指数级增长，最终超出模型的上下文窗口限制。

痛点四：成本黑盒

不同模型的定价差异很大，而且每次调用的 token 数量难以预估。没有精确的成本追踪，很难做预算控制和优化决策。

agent-io 的设计理念

面对这些痛点，我们设计了 agent-io，核心原则是：

1. 抽象统一，实现灵活

```rust
/// 所有 LLM 都实现这个统一接口
#[async_trait]
pub trait BaseChatModel: Send + Sync {
    /// 获取模型名称
    fn model(&self) -> &str;

    /// 获取提供商名称
    fn provider(&self) -> &str;

    /// 同步调用
    async fn invoke(
        &self,
```

```
        messages: Vec<Message>,
        tools: Option<Vec<ToolDefinition>>,
        tool_choice: Option<ToolChoice>,
    ) -> Result<ChatCompletion, LlmError>;

    /// 流式调用
    async fn invoke_stream(
        &self,
        messages: Vec<Message>,
        tools: Option<Vec<ToolDefinition>>,
        tool_choice: Option<ToolChoice>,
    ) -> Result<ChatStream, LlmError>;
}
```

通过这个 trait，无论底层是 GPT-5 还是 Claude，业务代码都保持一致。

2. 工具即函数

用 Rust 的类型系统和闭包，让工具定义变得自然：

```
// 定义一个天气工具，类型安全
let weather_tool = FunctionTool::new(
    "get_weather",
    "获取指定城市的天气信息",
    json!({
        "type": "object",
        "properties": {
            "location": {
                "type": "string",
                "description": "城市名称，如 '北京'"
            }
        },
        "required": ["location"]
    })
    .as_object()
    .unwrap()
    .clone(),
    |args: WeatherArgs| {
        Box::pin(async move {
            // 实际业务逻辑
            Ok(format!("{} 当前天气：晴，25° C", args.location))
        })
    },
);
```

3. 事件驱动架构

Agent 执行过程是流式的，我们用事件来描述每个阶段：

```rust
pub enum AgentEvent {
    /// 文本输出
    Text(TextEvent),
    /// 思考过程（Claude 支持）
    Thinking(ThinkingEvent),
    /// 工具调用
    ToolCall(ToolCallEvent),
    /// 工具结果
    ToolResult(ToolResultEvent),
    /// 最终响应
    FinalResponse(FinalResponseEvent),
    /// 错误
    Error(ErrorEvent),
    /// 步骤开始
    StepStart(StepStartEvent),
    /// 步骤完成
    StepComplete(StepCompleteEvent),
}
```

这让开发者可以精确控制 Agent 的执行流程，实现进度展示、日志记录等能力。

4. 智能上下文压缩

针对上下文爆炸问题，我们引入了 **Ephemeral Tool** 概念：

```rust
// 工具结果用完即销毁，不占用上下文
let search_tool = FunctionTool::new(...)
    .with_ephemeral(EphemeralConfig::Single);
```

这类工具的输出在下一轮迭代前会被自动清理，显著降低 token 消耗。

从零打造 Agent SDK：核心实现解析

接下来，我们深入探讨 agent-io 的核心实现，希望能给想要自己造轮子的开发者一些启发。

第一步：定义统一的消息模型

不同 LLM 的消息格式不同，我们需要一个统一的内部表示：

```rust
/// 统一的消息类型
pub enum Message {
    /// 用户消息
```

```
    User(UserMessage),
    /// 助手消息
    Assistant(AssistantMessage),
    /// 系统消息
    System(SystemMessage),
    /// 工具消息
    Tool(ToolMessage),
}

/// 用户消息
pub struct UserMessage {
    pub role: String,
    pub content: Vec<ContentPart>,
}

/// 内容部分（支持文本和多模态）
pub enum ContentPart {
    Text { text: String },
    Image { image_url: String },
    // ... 其他类型
}
```

这样，无论调用哪个 LLM，我们都先把外部格式转换为内部格式，再由各 Provider 转换为自己的 API 格式。

第二步：实现 Agent 核心循环

Agent 的核心是一个循环：调用 LLM → 处理工具调用 → 再调用 LLM → ... 直到得到最终答案。

```
impl Agent {
    fn execute_loop(&self) -> impl Stream<Item = AgentEvent> + '_ {
        async_stream::stream! {
            let mut step = 0;

            loop {
                // 1. 检查迭代次数
                if step >= self.config.max_iterations {
                    yield AgentEvent::Error(...);
                    break;
                }

                // 2. 清理 ephemeral 消息
                self.destroy_ephemeral_messages();

                // 3. 构建请求消息
                let messages = self.build_messages();

                // 4. 调用 LLM
                let completion = self.llm.invoke(messages, ...).await?;

                // 5. 处理响应
```

```
                if completion.has_tool_calls() {
                    // 执行工具，继续循环
                    for tool_call in &completion.tool_calls {
                        let result = self.execute_tool(tool_call).await;
                        yield AgentEvent::ToolResult(result);
                    }
                    step += 1;
                    continue;
                }

                // 6. 返回最终结果
                yield AgentEvent::FinalResponse(...);
                break;
            }
        }
    }
}
```

第三步：优雅的错误处理与重试

LLM API 经常遇到限流，需要内置重试机制：

```
async fn call_llm_with_retry(
    llm: &dyn BaseChatModel,
    messages: Vec<Message>,
    tools: Option<Vec<ToolDefinition>>,
    tool_choice: Option<ToolChoice>,
) -> Result<ChatCompletion> {
    let max_retries = 3;
    let mut delay = Duration::from_millis(100);

    for attempt in 0..=max_retries {
        match llm.invoke(messages.clone(), ...).await {
            Ok(completion) => return Ok(completion),
            Err(LlmError::RateLimit) if attempt < max_retries => {
                tokio::time::sleep(delay).await;
                delay *= 2;  // 指数退避
            }
            Err(e) => return Err(e.into()),
        }
    }

    Err(Error::Agent("Max retries exceeded".into()))
}
```

第四步：Token 消耗追踪

精确追踪每次调用的 token 消耗和成本：

```rust
/// 使用量汇总
pub struct UsageSummary {
    /// 总 token 数
    pub total_tokens: u64,
    /// 按模型分组的使用量
    pub by_model: HashMap<String, ModelUsage>,
    /// 估算成本（美元）
    pub estimated_cost: f64,
}

impl Agent {
    /// 获取当前会话的使用量统计
    pub async fn get_usage(&self) -> UsageSummary {
        self.usage.read().await.clone()
    }
}
```

第五步：Builder 模式提升易用性

一个优秀的 SDK 应该让常见场景变得简单：

```rust
let agent = Agent::builder()
    .with_llm(Arc::new(llm))
    .tool(weather_tool)
    .tool(search_tool)
    .system_prompt("你是一个专业的助手")
    .max_iterations(100)
    .build()?;

// 流式查询
let stream = agent.query_stream("今天北京天气如何？").await?;
pin_mut!(stream);

while let Some(event) = stream.next().await {
    match event {
        AgentEvent::Text(e) => print!("{}", e.content),
        AgentEvent::ToolCall(e) => println!("\n[调用工具: {}]", e.name),
        AgentEvent::FinalResponse(e) => println!("\n{}", e.content),
        _ => {}
    }
}
```

Rust：为什么选择它？

你可能会问：为什么用 Rust 而不是 Python 或 TypeScript？

1. 性能与安全兼得

- **零成本抽象**：trait、泛型在编译期展开，运行时无额外开销
- **内存安全**：无 GC 停顿，适合长时间运行的服务
- **类型系统**：编译期捕获大量错误，重构更有信心

2. 异步运行时成熟

Tokio 提供了生产级的异步运行时，处理并发请求游刃有余：

```rust
#[tokio::main]
async fn main() {
    // 轻松处理数万并发连接
}
```

3. 生态完善

- `reqwest`：成熟的 HTTP 客户端
- `serde`：强大的序列化框架
- `tracing`：结构化日志和追踪
- `tower`：中间件和服务抽象

4. 跨语言调用

通过 FFI，Rust SDK 可以被 Python、Node.js、Go 等语言调用，真正实现一次编写，到处使用。

已支持的能力

目前 agent-io 已经支持：

能力	状态
多模型支持	OpenAI、Anthropic、Google、Groq、Mistral、DeepSeek、Ollama、OpenRouter
工具调用	Function Calling，支持依赖注入
流式响应	事件驱动的实时输出

能力	状态
上下文压缩	Ephemeral Tool 自动清理
Token 追踪	使用量和成本统计
重试机制	指数退避，自动处理限流

结语：基础设施决定上限

在 AI 应用快速迭代的今天，我们常被问到：**Agent 的未来是什么？**

我们认为，Agent 的能力上限，不仅取决于模型，更取决于基础设施的成熟度。就像操作系统之于应用软件，Agent SDK 之于 AI 应用，都是不可或缺的底层支撑。

agent-io 是我们的一小步尝试。它还远不完美：

- 需要更丰富的工具生态
- 需要更智能的上下文管理策略
- 需要更多生产场景的验证

> ***但我们相信，开源的力量会让它变得更好。***

欢迎一起贡献 🤝

agent-io 是一个开源项目，我们诚挚邀请社区开发者一起参与：

GitHub: https://github.com/lispking/agent-io

你可以：

- 🐛 提交 Issue 报告 Bug 或建议新功能

- 🔧 贡献代码，实现新的 LLM 适配器
- 📖 完善文档和示例
- 💬 分享使用场景和最佳实践

让我们一起，为 Agent 时代打造更好的基础设施！